

System · Infra Engineer

이상혁

Lee Sang Hyeok

Linux 서버와 AWS를 직접 구성하고, 장애를 내서 복구를 확인합니다.

01 **failstop** 03
모의 거래소의 AWS 이중화와 DB 페일오버 시험

02 **keep-nginx-alive** 10
CTF에서 nginx를 다운 없이 유지한 방어 구조

03 **high-throughput-image-pipeline** 13
t3.micro 한 대의 이미지 수집 처리량 최적화

04 **ai-lunch-curator** 14
식단표 수집, Teams 알림, CI/CD 배포 자동화

05 **sangkwon-platform-poly** 15
EC2 두 대 롤링 배포와 서버 상태 DB 이전

Email kljl6773@gmail.com

GitHub github.com/SanghyeokLee-KR

Web sanghyeoklee-kr.github.io

Backend — Infra

백엔드 개발에서 시작해, 지금은 서버 운영과 장애 대응을 공부합니다

처음에는 웹 서비스 기능을 만드는 백엔드 프로젝트를 주로 했습니다. 만든 서비스를 직접 배포하면서 서버가 죽거나 요청이 몰릴 때의 동작을 확인할 일이 많아졌고, 지금은 Linux 서버와 AWS 구성을 직접 설정한 뒤 장애를 내서 복구되는지 확인하는 작업을 주로 합니다.

01

Linux 서버를 직접 설정합니다

계정과 권한, 로그, 방화벽, 디스크를 직접 설정했습니다. RAID1에서 디스크를 빼고 교체해 봤고, 디스크 용량을 일부러 채워 경보가 오는지 확인했습니다.

02

장애를 내서 복구를 확인합니다

주 DB를 강제로 끄고, 예비 DB가 승격되는 동안 주문이 사라지는지 대조했습니다. 공격/방어 CTF에서는 경기 내내 채점 기준 다운이 한 번도 없었습니다.

03

관제와 배포를 자동화했습니다

Prometheus와 Grafana로 지표를 보고, 경보가 울리면 AI가 원인을 정리해 Teams로 보고하게 했습니다. 배포는 GitHub Actions로 자동화했습니다.

지금까지 해 온 프로젝트

2024	2025	2026
BookRidge · DevHub	DeliveryPro · SafetyNevi	polytalk · kopo-ecommerce · keep-nginx-alive · ai-lunch-curator · image-pipeline · 하모니 · sangkwon · **failstop**

주로 쓰는 기술

서버 운영	Linux · systemd · nftables · Docker · Nginx · HAProxy
클라우드	AWS EC2 · ALB · Auto Scaling · CloudFront · SSM · S3
관제·자동화	Prometheus · Grafana · Alertmanager · GitHub Actions
데이터	PostgreSQL · Patroni · Oracle · Redis
개발	Java · Spring Boot · Python · FastAPI · TypeScript
학력	인하공업전문대학교 정보통신공학과 학사(전공심화) 2026.02
자격·수상	정보처리기사 · 네트워크관리사 2급 · 폴리텍 프로젝트작품 경진대회 은상

1인 프로젝트 · 2026년 7월 ~ 10월

직접 만든 모의 거래소를 AWS에 이중화하고, DB 장애와 부하를 시험했습니다

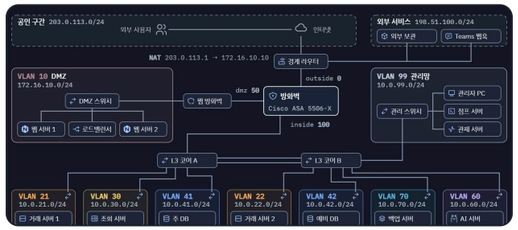
직접 만든 모의 거래소를 운영 대상 서비스로 두고 인프라를 구성했습니다. 제 PC의 Docker에서 장애 시험을 먼저 하고, Packet Tracer로 망을 설계하고, 가상 머신으로 서버를 다시 구성한 다음 AWS로 옮겼습니다.



1단계

로컬 Docker 시험

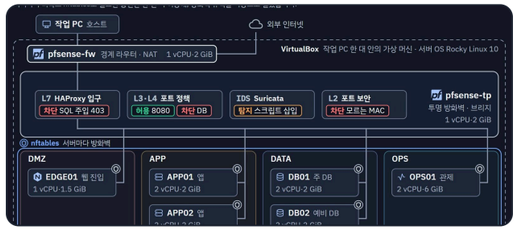
Docker로 거래소를 띄우고 장애와 부하를 넣어 봤습니다.



2단계

네트워크 설계

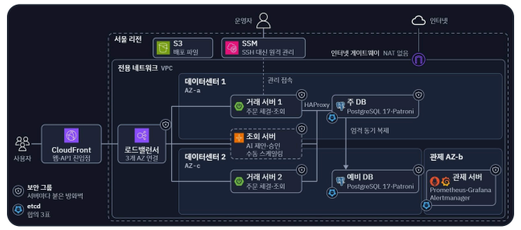
Packet Tracer로 필요한 통신만 허용하도록 망을 설계했습니다.



3단계

가상 머신 구성

가상 머신에 서버 여섯 대를 올려 방화벽과 백업을 구성했습니다.



4단계

AWS 배포

두 가용 영역에 나눠 배포하고, DB 장애와 서버 증설을 시험했습니다.

179회

일부러 낸 장애

제 PC의 Docker에서, 이어받기와 DB 승격은 30초 이내

23.0초

주 DB가 꺼지고 예비 DB가 넘겨받기까지

AWS 서울 리전, 대조한 주문 196건 중 유실 0건

4분 44초

CPU 경보부터 조회 서버 합류까지

AI 제안을 승인하고 67초 뒤 합류

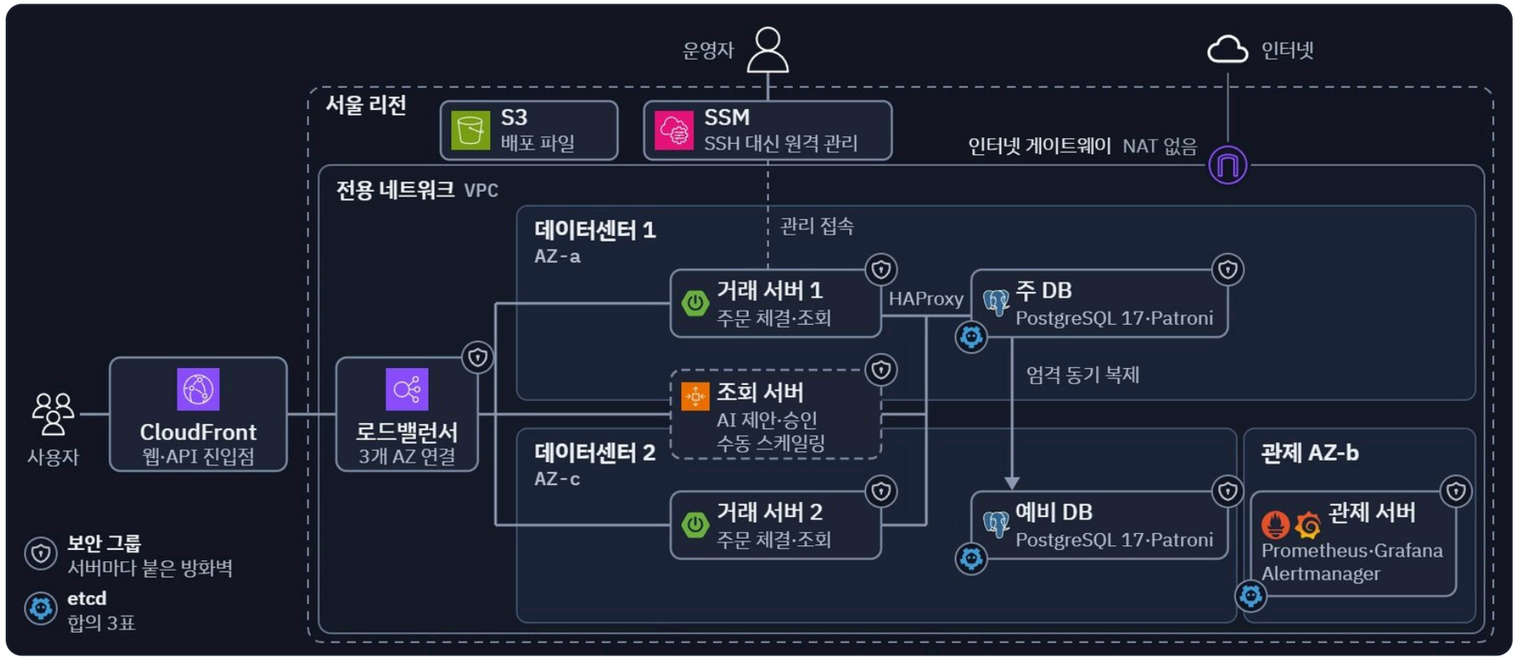
36.7만 원

한 달 예상 운영비

세금 포함, 예산 40만 원 이내

AWS 서울 리전

거래 서버와 DB를 두 가용 영역에 나눠 배치했습니다



2026년 10월에 운영한 구성입니다. 발표 후에는 비용이 나가지 않게 자원을 모두 정리했습니다.

01 **거래 서버와 DB는 두 데이터센터에**

가용 영역 두 곳에 한 대씩 나눠 두었습니다. 한쪽 서버가 멈추면 다른 쪽이 거래를 이어받습니다.

02 **조회 서버는 평소에 꺼 둡니다**

부하가 올라갈 때만 켜지게 해서 상시 운영 비용을 줄였습니다.

03 **DB는 Patroni 엄격 동기 복제**

주 DB가 멈추면 같은 내용을 가진 예비 DB만 새 주 DB가 됩니다. 커밋을 확인해 준 주문은 예비 DB에도 남아 있습니다.

04 **서버 접속은 SSM으로**

SSH 대신 SSM으로 접속했고, 배포 파일은 S3에 올려 두었습니다.

05 **예산은 한 달 40만 원**

평소 사용량으로 계산한 예상 비용은 세금 포함 **약 36.7만 원**입니다.

제 PC의 Docker

AWS에 올리기 전에, 로컬 Docker에서 장애 시험을 179회 진행했습니다



주 DB를 강제로 끄자 약 23초 뒤 예비 DB가 새 주 DB가 되고, 거래 서버의 연결도 새 주 DB로 옮겨 갑니다. 확인해 준 주문은 사라지지 않았습니다.

장애	횟수	가장 오래 걸린 복구
DB 연결 끊기	28	13.5초
거래 서버 강제 종료	48	12.9초
거래 서버 멈춤	28	13.4초
네트워크 끊기	17	11.1초
주 DB 강제 종료	15	27.0초
DB 지연 등 그 밖의 시험	43	—

시험 중 찾아서 고친 문제

01 **DB 연결이 끊기면 리더 없이 멈췄습니다**

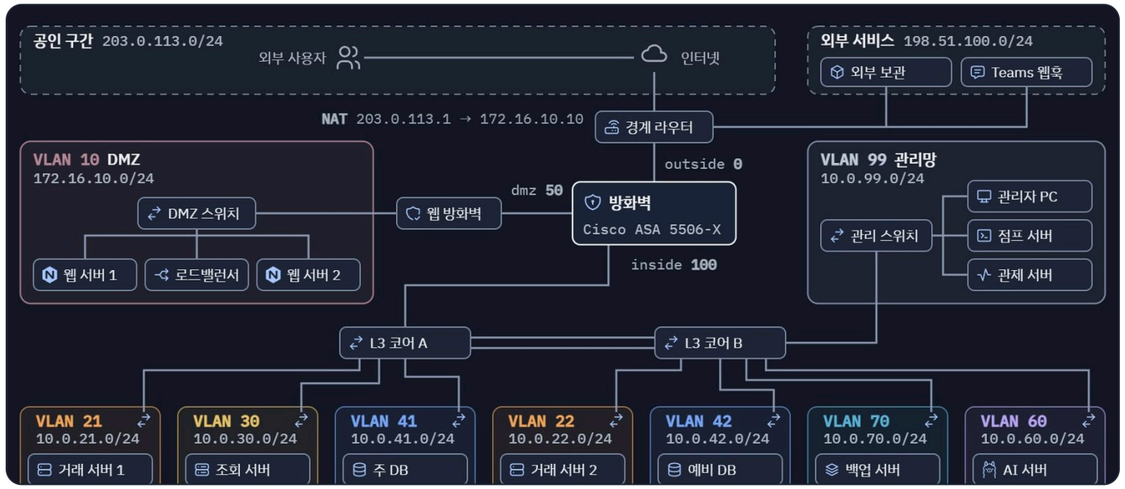
트랜잭션을 연 채 멈춘 세션이 잠금을 쥐고 있었습니다. DB에 10초 제한과 keepalive를 걸자 고친 직후 **13.9초** 만에 이어받았고, 이후 28회 반복에서는 길어도 13.5초였습니다.

02 **호가 집계가 주문 처리를 막았습니다**

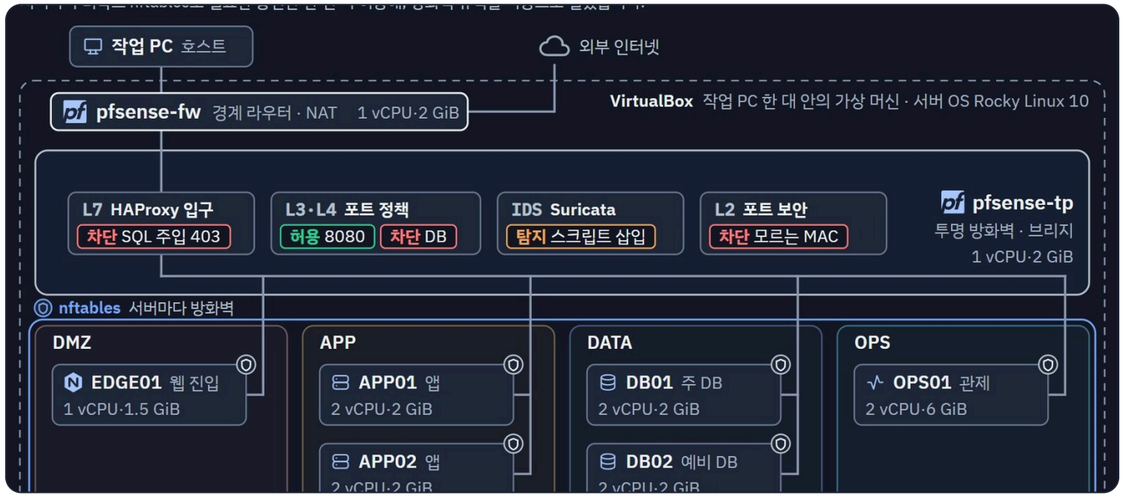
집계 결과를 1초 동안 캐시하자, 놓치지 않고 접수한 주문이 **초당 40건에서 80건**으로 늘었습니다.

Packet Tracer와 가상 머신

망을 구간별로 나누고, 계층마다 방화벽을 두었습니다



망 설계 · 외부 웹 요청은 DMZ에서 받고, 거래 서버와 DB는 내부망에 두었습니다. 관리자도 점프 서버를 거쳐야 서버에 접속할 수 있습니다.



가상 머신 실습망 · 서버 설정은 그대로 두고 서버 앞에 투명 방화벽(pfSense 브리지)을 넣었습니다. 서버 안에는 nftables를 한 겹 더 두었습니다.

입구에서 TLS를 풀고 요청을 검사합니다 HAProxy

TLS를 종료해 요청 내용을 확인하고, SQL 주입 시험 요청은 403으로 돌려보냈습니다.

필요한 포트만 열어 둡니다 pfSense

웹 서버는 앱 서버의 8080 포트로만 갈 수 있고, DB로 바로 가는 길은 막혀 있습니다.

침입 시도를 탐지합니다 Suricata

웹과 앱 사이 트래픽에서 스크립트 삽입 시도를 탐지해 관제 서버로 경보를 보냈습니다.

서버마다 기본 차단 nftables

기본은 모두 막고 필요한 포트만 엽니다. 규칙 원본 하나를 서버 여섯 대에 똑같이 적용했습니다.

온프레미스 AI 서버와 Teams

경보가 울리면 AI가 원인을 정리하고 조치를 제안합니다



2026년 10월 10일 AWS 부하 시험 때의 순서를 장면으로 다시 그렸습니다.

- 03:02:12 CPU 경보가 울리고, Teams로 바로 알림이 갑니다.
- 03:03:57 AI가 지표와 서버 수, 최근 조치를 모아 조회 서버를 한 대 늘리자고 제안합니다.
- 03:05:49 AI가 제시한 근거를 확인하고 승인합니다.
- 03:06:56 승인하고 67초 뒤 조회 서버가 합류합니다.
- 03:07 경보가 풀리고, 장애 보고서가 Teams로 옵니다.

01 **AI는 제안만 합니다**

서버를 바꾸는 일은 제가 승인해야 실행됩니다. 허용해 둔 조치 다섯 가지 밖의 제안은 코드가 먼저 걸러 냅니다.

02 **DB 전환은 AI에게 맡기지 않았습니다**

되돌리기 어려운 작업이라 Patroni의 페일오버 규칙에 맡겼습니다.

AWS와 가상 머신에서

DB 장애와 OS 삭제 뒤에도 데이터가 남는지 확인했습니다

DB 전원을 끊었을 때 AWS

23.0초

주 DB 전원을 끊자 예비 DB가 새 주 DB로 승격됐습니다. 장애 4초 전까지 1분 동안 커밋된 주문 196건을 새 주 DB와 하나씩 대조했고, **사라진 주문은 없었습니다.**

OS를 통째로 지웠을 때 가상 머신

15분 22초

DB 서버의 OS와 데이터 디스크를 지운 뒤, 다시 설치하고 백업으로 되살리기까지 걸린 시간입니다. 따로 떠 둔 DB 백업으로 복원했을 때도 테이블 109개가 원래 DB와 모두 같았습니다.

평소에 해 두는 일

앱은 로그인할 수 없는 전용 계정으로 돌립니다.

로그는 재부팅 뒤에도 남도록 디스크에 보관합니다.

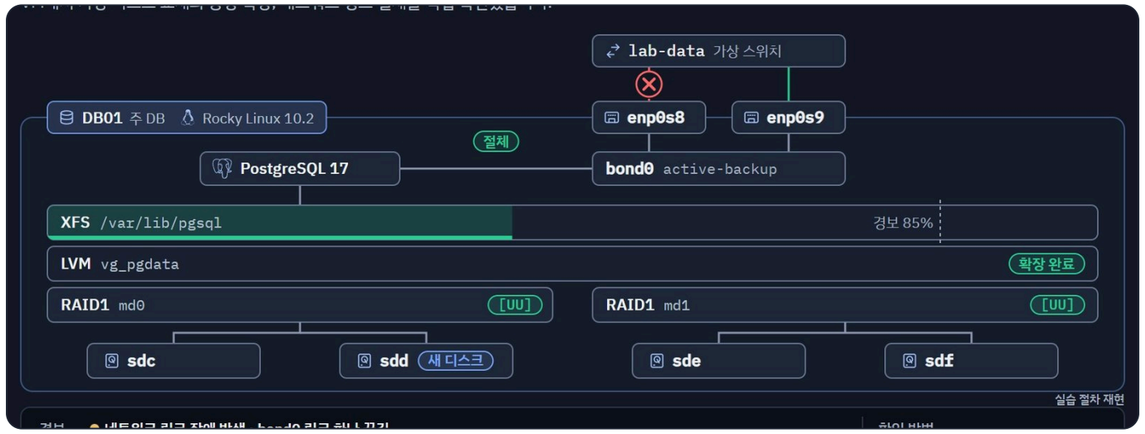
방화벽 규칙을 잘못 넣으면 90초 뒤 저절로 되돌아갑니다.

디스크 여유가 15% 아래로 떨어지면 경보가 옵니다.

디스크 하나가 고장 나도 RAID1로 버팁니다.



백업과 복구 기록 · 복원한 DB 대조와 OS 재설치 뒤 복구 화면



서버 안쪽 이중화 · RAID1 교체, LVM 증설, NIC 본딩 전환

환경별 시험 결과와 운영하면서 정한 기준

시험	환경	결과
일부러 낸 장애	제 PC의 Docker	179회, 이어받기·DB 승격 30초 이내
주 DB 전원 차단	AWS	23.0초 만에 전환, 주문 유실 0건
부하 시 조회 서버 증설	AWS	경보부터 합류까지 4분 44초
호가 집계 캐시	제 PC의 Docker	접수 주문 초당 40 → 80건
OS 삭제 후 복구	가상 머신	15분 22초

GitHub 저장소 발표 자료 시연 영상(7분)

01 **이중화와 함께 경보·보고·복구 절차를 만들었습니다**

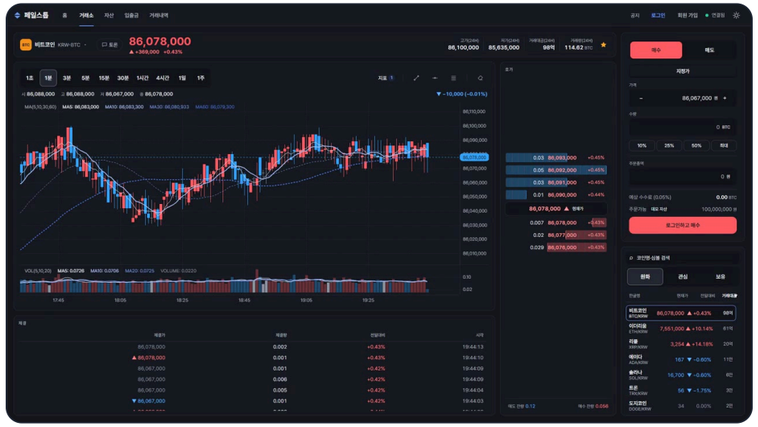
처음에는 이중화만 구성하면 된다고 봤습니다. 장애를 내 보니 경보, Teams 보고, 백업 복원 절차까지 있어야 대응할 수 있어서 함께 구성하고 시험했습니다.

02 **자동 조치의 범위를 나눴습니다**

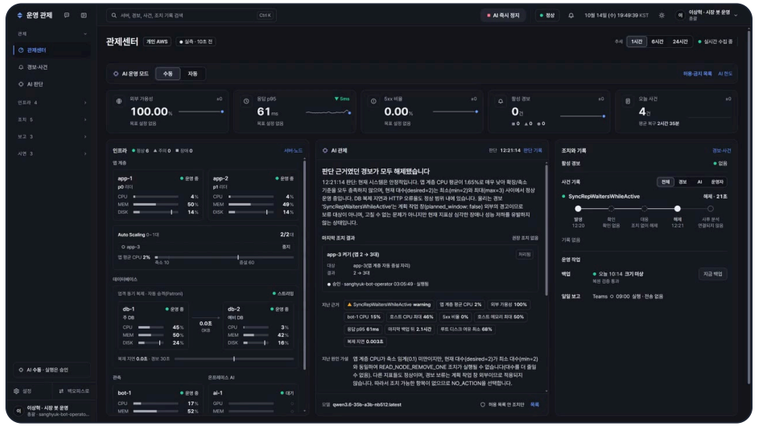
DB 전환은 Patroni 규칙에, 조회 서버 증설은 AI 제안과 운영자 승인에 맡겼습니다. AI가 제안할 수 있는 조치는 허용 목록 안으로 제한했습니다.

03 **측정 환경과 결과를 함께 기록했습니다**

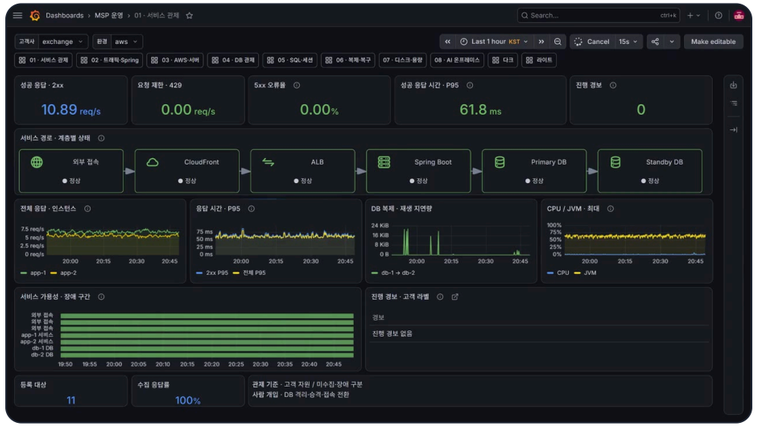
시험마다 제 PC의 Docker, 가상 머신, AWS 중 어디서 했는지와 회차, 실행한 명령과 화면을 함께 남겼습니다.



거래소 · 시장 봇이 주문을 주고받는 화면



관제센터 · 서버와 DB 상태, AI 운영 모드



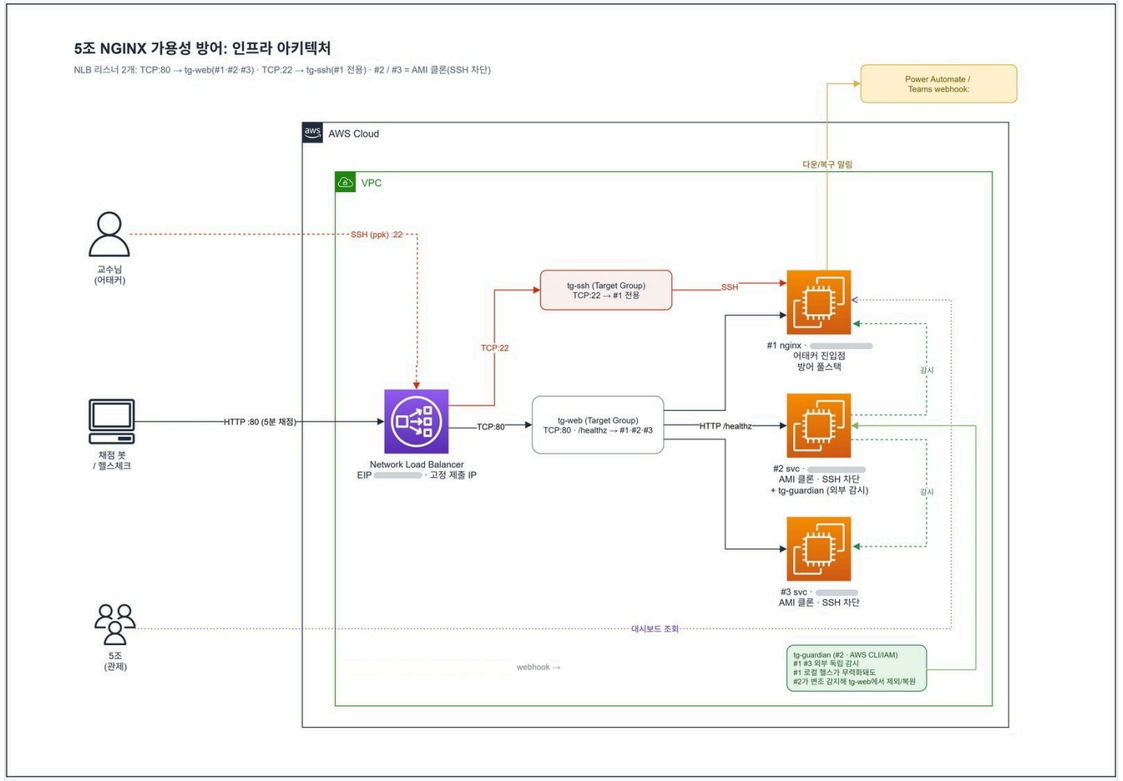
Grafana · 요청, 오류, 응답 시간

3인 팀 조장 · 2026년 5월 · 공격/방어 CTF

공격자를 차단할 수 없는 CTF에서, 사흘간 채점 기준 다운 0회를 기록했습니다

교수님이 root 권한으로 들어와 nginx를 끄고 포트를 빼앗는 경기였습니다. 규칙상 공격자를 차단할 수 없어서, nginx가 꺼지면 바로 다시 띄우고 그래도 안 되면 그 서버를 로드밸런서에서 빼는 구조로 방어했습니다.

제가 한 일 복구 · 감시 · 페일오버 방어 구조 전체를 설계하고 만들었습니다. 경기 기간 새벽에는 세 명이 Discord로 교대해 불침번을 섰습니다.



공격자가 들어오는 서버는 한 대뿐이고, 같은 구성의 서버 두 대가 로드밸런서 뒤에서 함께 채점을 받습니다.

0회
채점 기준 서비스 다운
5월 29일 ~ 31일 채점 기록 기준

2초
다운에서 복구까지
Teams 알림 기록 기준

5겹
서로를 살리는 복구 장치
서로 다른 장치 다섯 가지

nginx를 다섯 가지 장치로 되살리고, 안 되면 그 서버를 NLB에서 뺍니다



01 **다섯 가지 복구 장치**

systemd 재시작(2초), 감시 스크립트 둘(5초), cron(1분), 무결성 감시(10초), systemd 밖의 독립 감시(3초)가 nginx와 꺼진 장치를 함께 되살립니다. 여러 장치가 동시에 재시작하다 충돌하지 않도록, flock으로 한 번에 하나만 실행되게 했습니다.

02 **페이지가 오염되면 대상 그룹에서 빠집니다**

헬스체크 스크립트가 2초마다 본문을 검사해, 오염되면 /healthz가 503을 돌려줍니다. 로드밸런서는 그 서버를 빼고 #2·#3으로 채점 요청을 보냅니다.

03 **공격자의 명령은 auditd로 기록했습니다**

auditd로 명령을 모으고, 로그인 · 명령 · 다운과 복구는 Power Automate를 거쳐 Teams 세 채널로 보냈습니다.

세 번의 공격을 받았고, 채점 기준 다운 없이 대응했습니다



경기 중 직접 만든 관제 화면입니다. 접속자, 복구 장치 상태, 공격자의 명령이 실시간으로 올라옵니다.

1차 **감시 스크립트와 nginx를 끄고 서버를 둘러봤습니다**

남아 있던 다른 복구 장치가 되살렸고, Teams 알림 기준 다운 감지 2초 뒤 복구됐습니다.

2차 **apache2를 설치해 80번 포트를 빼앗았습니다**

#1이 스스로 빠지고 #2·#3이 채점을 받는 사이, 팀원이 들어가 apache2를 정리했습니다. 채점은 끊기지 않았습니다.

3차 **nginx가 다시 켜지지 못하게 막았습니다**

systemctl mask로 막힌 nginx를 감시 스크립트가 다시 풀고 띄웠습니다.

> **채점이 끊기지 않은 데는 #1의 복구 장치보다 로드밸런서와 복제 서버 #2·#3의 역할이 컸습니다. 문제가 생긴 서버를 대상 그룹에서 빼고 다른 서버로 넘기는 방식은 이후 failstop의 증설·교체 구조로 이어졌습니다.**

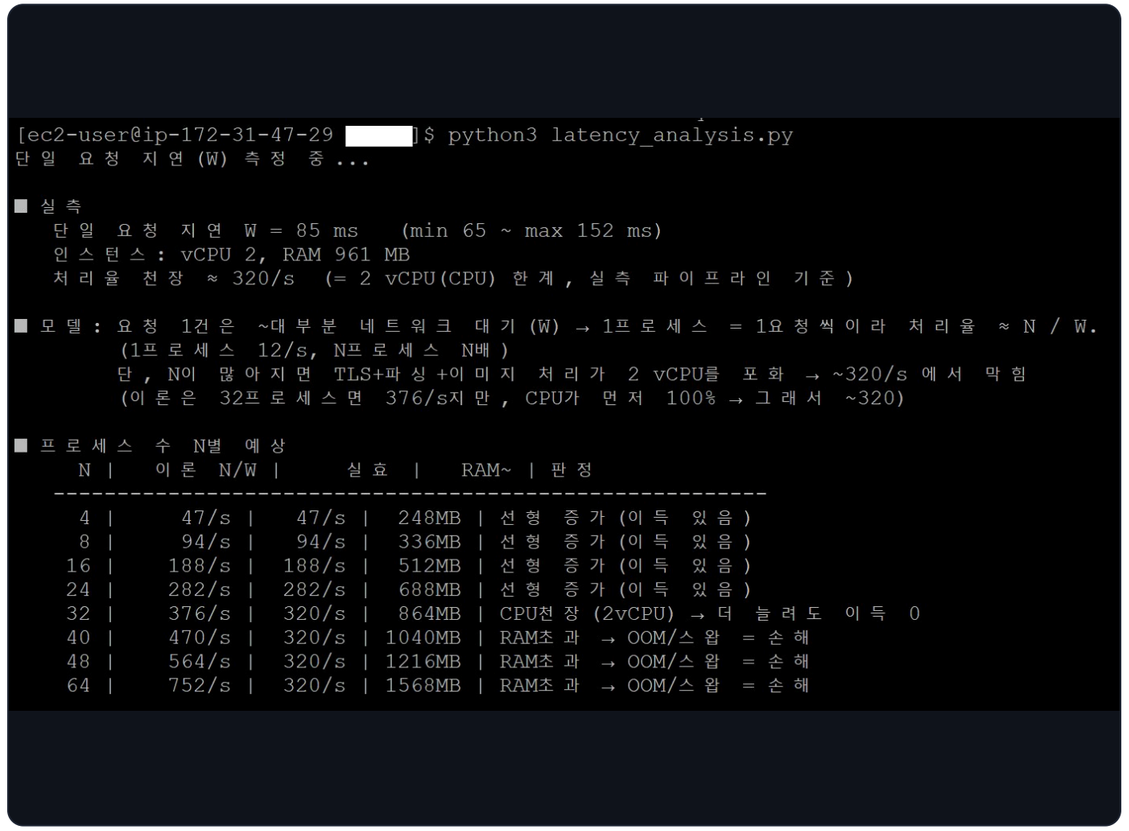
3인 팀 · 2026년 6월 · 팀 제출본으로 채택

t3.micro 한 대에서 이미지 수집 처리량을 측정하고, CPU 병목을 확인했습니다

t3.micro(vCPU 2개, 메모리 1GB) 한 대로 상품 이미지를 최대한 빨리 모으는 과제였습니다. 유효한 상품이 드문드문 있어서 200장을 모으려면 4천 페이지 넘게 확인해야 했습니다. 세 명이 각자 만들어 속도를 겨뤘고, 가장 빠른 제 구현이 팀 제출본이 됐습니다.

처음에는 네트워크가 병목일 거라고 예상했습니다. 요청 하나의 지연을 재고 동시성을 올려 보니 차단은 한 번도 없었고 **CPU가 먼저 100%에 닿았습니다.** 그래서 프로세스를 늘리는 대신 코루틴으로 동시성을 높였고, CPU는 100%까지 쓰면서 메모리는 최대 47%였습니다.

방식	200장 걸린 시간	처리 속도	메모리 최대
코루틴(프로세스 2 × 128)	16.14초	284페이지/초	47%
프로세스 32개	20.12초	223페이지/초	86%
프로세스 46개	16.45초	274페이지/초	99%



요청 하나에 약 85ms가 걸렸고, 동시성을 올려도 초당 약 320페이지에서 CPU가 먼저 포화됐습니다.

3인 팀 조장 · 2026년 6월

식단표 수집, Teams 알림, 배포를 GitHub Actions로 자동화했습니다

매주 월요일 GitHub Actions가 식단표를 모으고, AI가 메뉴를 정리해 DB에 쌓습니다. 평일 아침에는 그날 점심이 Teams로 옵니다. 저는 전체 구조와 웹 · DB, 그리고 Docker · EC2 · Nginx 배포와 CI/CD를 맡았고, 크롤링과 AI 프롬프트, Teams 발송은 팀원이 맡았습니다.

01 **테스트를 통과해야만 배포됩니다**

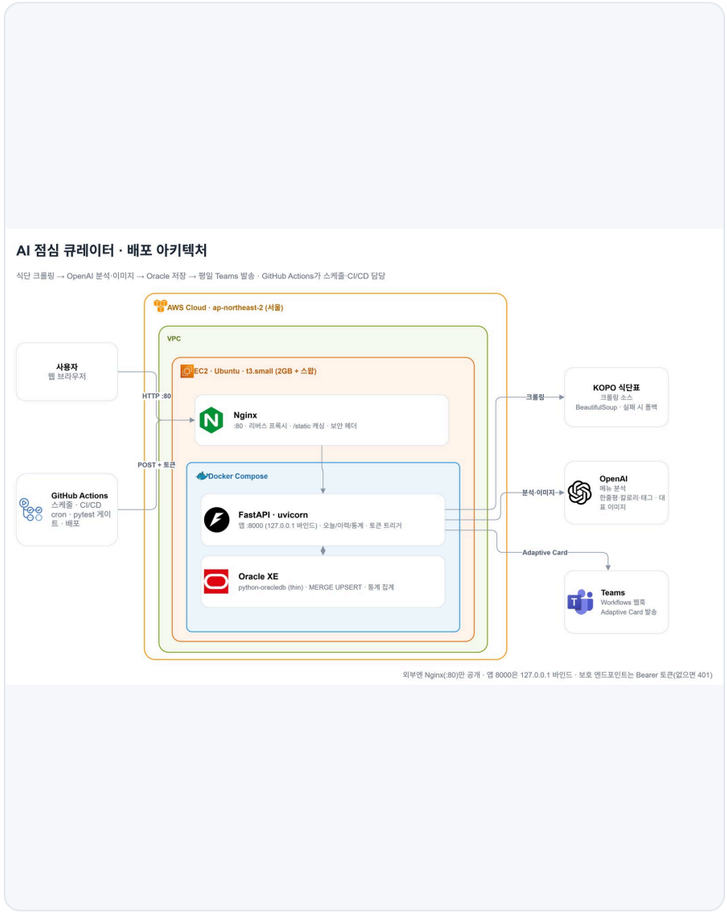
main에 push하면 pytest 23개가 먼저 돌고, 모두 통과해야 EC2에 배포합니다.

02 **비용이 드는 주소는 토큰으로 막았습니다**

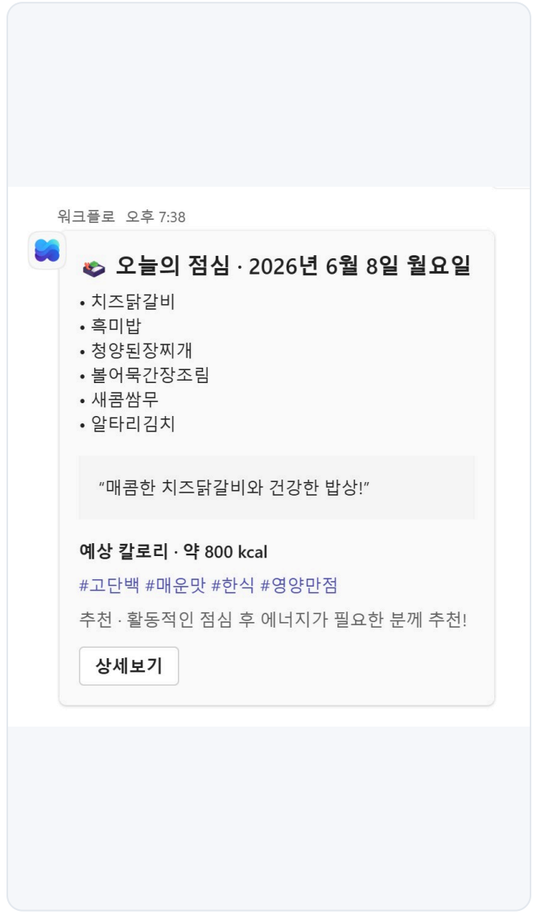
OpenAI를 부르는 수집 주소와 발송 주소는 Bearer 토큰이 없으면 401로 돌려보냅니다.

03 **같은 메뉴 그림은 다시 만들지 않습니다**

메뉴 구성을 해시로 묶어 두어, 같은 구성이면 이미 만든 그림을 다시 쓰고 AI 호출을 줄였습니다.



배포 구조 · GitHub Actions가 EC2의 컨테이너를 호출합니다.



평일 아침 Teams로 오는 알림입니다.

4인 팀 팀장 · 2026년 7월

EC2 두 대를 한 대씩 교체하는 롤링 배포를 구성했습니다

서울시 상권 1,650곳의 매출과 유동인구, 점포 수를 지도에서 비교하는 서비스입니다. 저는 상권분석 · 지도 백엔드와 팀 공통 기반, 그리고 배포를 맡았습니다.

01 **서로 다른 가용 영역에 서버 두 대**

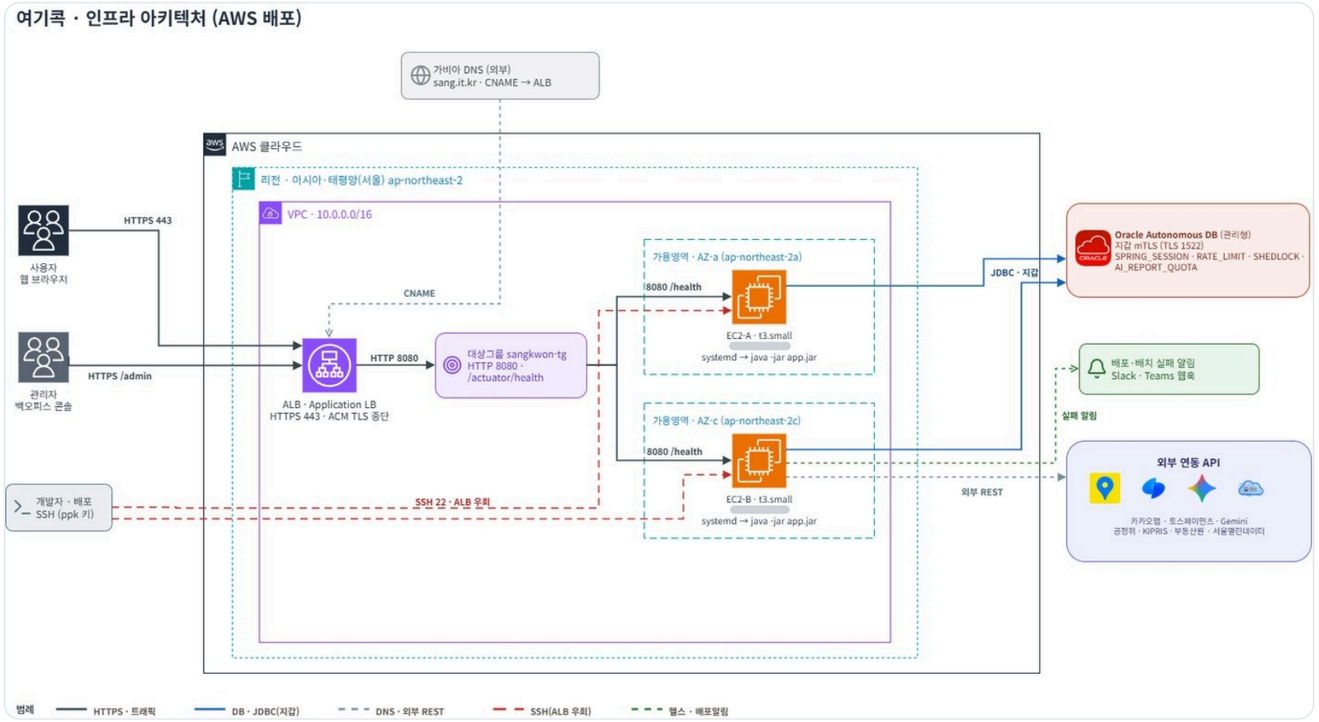
ALB가 ACM 인증서로 HTTPS를 받고, /actuator/health를 30초마다 확인해 응답하지 않는 서버를 대상에서 뺍니다.

02 **서버 상태는 DB에 둡니다**

로그인 세션(Spring Session JDBC), 스케줄러 락(ShedLock), 요청 제한 카운터, 데이터 재적재 상태를 DB로 옮겨 두 서버가 함께 씁니다.

03 **한 대씩 바꾸는 롤링 배포**

A를 먼저 바꾸고, 헬스체크를 통과해 대상에 돌아온 뒤에 B를 바꿉니다. 내려가는 서버는 진행 중인 요청을 최대 20초 기다렸다가 종료합니다.



가용 영역 a와 c의 EC2 두 대가 ALB 뒤에 있고, DB는 Oracle Autonomous DB입니다.

각 프로젝트의 자세한 내용은 GitHub 에 정리해 두었습니다.